

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250284776

Kind Code

A1

Publication Date

September 11, 2025

Inventor(s)

Assadipour; Pouria et al.

STREAMING FRAUD DETECTION USING BLOCKCHAIN

Abstract

Systems and methods for detecting fraudulent streaming activity. Streaming activity is posted to a blockchain by one or more DSPs. Blockchain streaming data is extracted from the blockchain and used as input in a machine learning model. The machine learning model takes the extracted blockchain data, along with additional inputs such as DSP trend pool and social pool inputs, and makes a determination regarding potentially fraudulent streaming activity.

Inventors: Assadipour; Pouria (Burnaby, CA), Martin; Evan (Port Moody, CA), Batey; Andrew (Vancouver, CA), Hayduk; Morgan (Toronto, CA)

Applicant: Beatdapp Software Inc. (Vancouver, CA)

Family ID: 84285117

Assignee: Beatdapp Software Inc. (Vancouver, CA)

Appl. No.: 19/219163

Filed: May 27, 2025

Related U.S. Application Data

parent US continuation 17805404 20220603 parent-grant-document US 12346413 child US 19219163

us-provisional-application US 63196600 20210603

Publication Classification

Int. Cl.: G06F21/10 (20130101); G06N20/20 (20190101); H04L9/00 (20220101); H04L9/32 (20060101)

U.S. Cl.:

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS [0001] This application is a continuation of U.S. patent application Ser. No. 17/805,404 (Attorney Docket No. BTPPP003) by Assadipour et al., filed Jun. 3, 2022, titled STREAMING FRAUD DETECTION USING BLOCKCHAIN, which application claims the benefit of priority under 35 U.S.C. § 119 (e) to U.S. Provisional Application No. 63/196,600 (BTPPP003P), titled “STREAMING FRAUD DETECTION USING BLOCKCHAIN,” filed Jun. 3, 2021, the entirety of which is incorporated in its entirety by this reference for all purposes.

TECHNICAL FIELD

[0002] The present disclosure relates to digital media, and specifically to tracking streaming media.

BACKGROUND

[0003] The music industry generates an estimated \$25 billion in revenue based on royalties. With the advent of the Internet, streaming technology makes it easy for listeners to listen to almost any song of their choosing. Artists usually work with music labels to distribute media and to help collect revenue based on royalties. These music labels distribute media through a variety of different mediums, including streaming platforms or digital service providers (DSPs), such as Spotify or Apple.

[0004] Although access to songs has been facilitated by DSPs, keeping track of all songs streamed or the amount of playback of certain songs has become an increasingly difficult problem to solve. In fact, as much as 25% of the activity on streaming platforms today is unlicensed. In addition, even for the licensed activity, up to 15% of total royalties remain uncollected annually. The DSPs claim that they lack the necessary data and technology to help figure out whose claims were legitimate, or even how to locate certain parties. In addition, the lack of an authoritative database that covers all existing music rights only adds to the problem. Since rights holders get paid for the number of plays that are accrued on tracks that they own partially or wholly, there is an incentive to artificially augment the play counts. Sometimes, bots are invoked to increase play counts, which is often against the rules of DSPs. Thus, there is a need for a reliable content identification technology that allows anyone to register, identify, and track creative works on the Internet.

SUMMARY

[0005] The following presents a simplified summary of the disclosure in order to provide a basic understanding of certain embodiments of the present disclosure. This summary is not an extensive overview of the disclosure and it does not identify key/critical elements of the present disclosure or delineate the scope of the present disclosure. Its sole purpose is to present some concepts disclosed herein in a simplified form as a prelude to the more detailed description that is presented later.

[0006] Aspects of the present disclosure relate to a system, method, and computer readable media for detecting fraudulent streaming activity. In one aspect, a system is provided. The system includes a processor and memory. The memory includes instructions for executing a method. The method begins by extracting streaming data from a blockchain. The streaming data corresponds to play count activity for streaming media. Next, the method includes transforming the extracted streaming data into a format ingestible by a trained machine learning model. The transformed streaming data is then input into the machine learning model. Last, the machine learning model determines that a particular stream is potentially fraudulent.

[0007] In some embodiments, the machine learning model takes into account user engagement data including one or more of the following: gyroscope orientation, battery percentage, current city,

track label, track distributor, and stream duration In some embodiments, the extracted streaming data comprises hashes instead of raw data for zero knowledge proofs. In some embodiments, the machine learning model includes multiple machine learning algorithms for different attributes of streaming media. In some embodiments, the data corresponds to other information including one or more of the following: device battery information, operating system, user interaction data t. In some embodiments, the machine learning model utilizes a social trend pool or a DSP trend pool as additional input. In some embodiments, the determination that the particular stream is potentially fraudulent is posted to a data store for direct or indirect access by DSPs.

[0008] Additional advantages and novel features of these aspects will be set forth in part in the description that follows, and in part will become more apparent to those skilled in the art upon examination of the following or upon learning by practice of the disclosure.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0009] The disclosure may best be understood by reference to the following description taken in conjunction with the accompanying drawings, which illustrate particular embodiments of the present disclosure. In the description that follows, like parts are marked throughout the specification and drawings with the same numerals, respectively. The drawing figures are not necessarily drawn to scale and certain figures may be shown in exaggerated or generalized form in the interest of clarity and conciseness.

[0010] FIG. 1 shows an example diagram of available user types' relationships, data, and functions in disclosed systems, in accordance with embodiments of the present disclosure.

[0011] FIG. 2 illustrates an example latency analysis from a song play request to serving fingerprinted content, in accordance with embodiments of the present disclosure.

[0012] FIG. 3 illustrates an example diagram of sharding, in accordance with embodiments of the present disclosure.

[0013] FIG. 4 shows a diagram of one example of how the music industry is connected, in accordance with embodiments of the present disclosure.

[0014] FIG. 5 shows a diagram of one example of the role of a basic administrator, in accordance with embodiments of the present disclosure.

[0015] FIG. 6 shows a diagram of one example of a basic data flow of an end-user, in accordance with embodiments of the present disclosure.

[0016] FIG. 7 shows a diagram of one example of a media file playback tracking network, in accordance with embodiments of the present disclosure.

[0017] FIG. 8 shows a system diagram of an example of a system for tracking media file playback, in accordance with embodiments of the present disclosure.

[0018] FIG. 9 shows a flowchart of a method for tracking media file playback using CDN architecture, in accordance with embodiments of the present disclosure.

[0019] FIG. 10 shows a flowchart of a method for tracking media file playback using non-CDN architecture, in accordance with embodiments of the present disclosure.

[0020] FIG. 11 illustrates an example non-CDN platform architecture for tracking media file playback, in accordance with embodiments of the present disclosure.

[0021] FIG. 12 illustrates an example of a transaction anatomy, in accordance with embodiments of the present disclosure.

[0022] FIG. 13 illustrates an example of data flow of a transaction, in accordance with embodiments of the present disclosure.

[0023] FIG. 14 illustrates another example of data flow of a transaction, in accordance with embodiments of the present disclosure.

[0024] FIG. **15** illustrates a flow diagram of an example non-CDN platform architecture, in accordance with embodiments of the present disclosure.

[0025] FIG. **16** illustrates another flow diagram of an example non-CDN platform architecture, in accordance with embodiments of the present disclosure.

[0026] FIG. **17** shows one example of a computer system, in accordance with embodiments of the present disclosure.

[0027] FIG. **18** shows one example of a linked list data structure, in accordance with embodiments of the present disclosure.

[0028] FIG. **19** shows a conceptual example of a blockchain, in accordance with embodiments of the present disclosure.

[0029] FIG. **20** shows an example of merklizing of a portion of a platform data set, in accordance with embodiments of the present disclosure.

[0030] FIG. **21** illustrates an example of a linked list, according embodiments of the present disclosure.

[0031] FIG. **22** illustrates a representation of an example streaming event in JSON format, according to embodiments of the present disclosure.

[0032] FIG. **23** is a block diagram showing component parts of a fraud detection system, in accordance with embodiments of the present disclosure.

[0033] FIG. **24** shows an example table containing suspected fraudulent streams, in accordance with embodiments of the present disclosure.

[0034] FIG. **25** is a diagram of components that go into a DSP Trend Pool, in accordance with embodiments of the present disclosure.

[0035] FIG. **26** is a diagram of components that go into a DSP Trend Pool plus Social Pool input, in accordance with embodiments of the present disclosure.

[0036] FIG. **27** is a diagram of components that go into a Social Pool, in accordance with embodiments of the present disclosure.

[0037] FIG. **28** illustrates a layered model, in accordance with embodiments of the present disclosure.

[0038] FIG. **29** illustrates a layered model with composable layers, in accordance with embodiments of the present disclosure.

[0039] FIG. **30A**-FIG. **30C** illustrate an example machine learning model architecture, in accordance with embodiments of the present disclosure.

DETAILED DESCRIPTION

[0040] Reference will now be made in detail to some specific examples of the disclosure including the best modes contemplated by the inventors for carrying out the disclosure. Examples of these specific embodiments are illustrated in the accompanying drawings. While the disclosure is described in conjunction with these specific embodiments, it will be understood that it is not intended to limit the disclosure to the described embodiments. On the contrary, it is intended to cover alternatives, modifications, and equivalents as may be included within the spirit and scope of the disclosure as defined by the appended claims.

[0041] For example, the techniques of the present disclosure will be described in the context of tracking request and play of media files, media file transmissions, cryptography, data storage, and media access validation. However, it should be noted that the techniques of the present disclosure apply to a wide variety of network transactions, collaborative environments, data structures, and different types of data. In the following description, numerous specific details are set forth in order to provide a thorough understanding of the present disclosure. Particular example embodiments of the present disclosure may be implemented without some or all of these specific details. In other instances, well known process operations have not been described in detail in order not to unnecessarily obscure the present disclosure.

[0042] Various techniques and mechanisms of the present disclosure will sometimes be described

in singular form for clarity. However, it should be noted that some embodiments include multiple iterations of a technique or multiple instantiations of a mechanism unless noted otherwise. For example, a system uses a processor in a variety of contexts. However, it will be appreciated that a system can use multiple processors while remaining within the scope of the present disclosure unless otherwise noted. Furthermore, the techniques and mechanisms of the present disclosure will sometimes describe a connection between two entities. It should be noted that a connection between two entities does not necessarily mean a direct, unimpeded connection, as a variety of other entities may reside between the two entities. For example, a processor may be connected to memory, but it will be appreciated that a variety of bridges and controllers may reside between the processor and memory. Consequently, a connection does not necessarily mean a direct, unimpeded connection unless otherwise noted.

[0043] As used herein, the term “platform” will refer to the platform systems and methods disclosed herein. Throughout the disclosure, the terms “platform” and “system” are used interchangeably. According to various embodiments, a blockchain music platform is provided. In some embodiments, a blockchain is defined as a decentralized, secure, and unalterable digital ledger or database that keeps a permanent record of all transactions. In some embodiments, on a blockchain ledger are “smart contracts,” which lay out the terms and costs of blockchain transactions. In some embodiments, the smart contracts can be amended, but all previous versions remain on the blockchain. Because of the complete history of transactions and changes to smart contracts, blockchain technology is inherently more transparent and secure than current systems of closed contracts and databases.

[0044] As used herein, the terms “sealers” and “validation nodes” will be used interchangeably and refer to any number of registered or trusted nodes used for validating transactions before adding the transactions to a blockchain. Examples of validation nodes can be DSPs, the platform, labels, and distributors.

[0045] In some embodiments, the blockchain music platform allows users to upload media files, e.g., songs, along with metadata associated with the media files, to the server or database. In some embodiments, once a request to upload a song is received, a blockchain transaction is created for the song and saved to the blockchain. In some embodiments, data such as song details, content access details, and stakeholders' stakes, as well as hundreds of other potential metadata fields, are also saved to the blockchain. In some embodiments, to ensure that data is immutable and the network is secure, transactions on the blockchain need to be validated before being finalized. In some embodiments, validation occurs only after at least 2 or more transaction validators (sealers), or authorized/trusted validation nodes, can validate the transactions.

[0046] In some embodiments, the blockchain streaming platform allows users to send metadata associated with the media files, to the server or database. In some embodiments, once a request to upload a song's data is received, a blockchain transaction is created for the song and saved to the blockchain. In some embodiments, data such as song details, content access details, and stakeholders' stakes, as well as hundreds of other potential metadata fields, are also saved to the blockchain. In some embodiments, to ensure that data is immutable and the network is secure, transactions on the blockchain need to be validated before being finalized. In some embodiments, validation occurs only after at least 2 or more transaction validators (sealers), or authorized/trusted validation nodes, can validate the transactions. In some embodiments, it only takes one signature by a validator to consider a transaction valid.

[0047] In some embodiments, there will be transaction fees to conduct transactions in the form of blockchain gas. In some embodiments, transactions will happen on a type of blockchain that doesn't have transaction fees. In some embodiments, transactions will be conducted on another protocol and be settled on the original protocol which could lead to a combination of gas fees. In some embodiments, tokens for the protocol will be used for payment for streaming data. In some embodiments, tokens will be used as payment to upload to the protocol.

[0048] In some embodiments, transactions are validated using a Proof of Authority (PoA) consensus algorithm. In other embodiments, transactions are validated using a Proof of Work (PoW) algorithm. In other embodiments, transactions are validated using a Proof of Stake (PoS) algorithm. However, according to various embodiments, PoA works better for the systems and methods disclosed for the following reasons. First, in PoA, only certain parties are allowed to validate transactions on the blockchain. Since the nodes that are allowed to seal blocks or validate transactions and identify which party they belong to are predefined, then the time needed for validation can be decreased. Second, transactions in PoA can be finalized as soon as they are processed by the blockchain, since any authorized sealer can cryptographically validate transactions. Consequently, blocks can be appended to the blockchain in any order as long as they are valid. Thus, another consequence of the PoA is that “uncle blocks” or “orphan blocks” are eliminated, since valid blocks do not get rejected simply due to the mechanism of the blockchain. Third, transaction throughput for PoA implementations are only limited by the hardware they run on. Thus, the systems disclosed can accommodate for millions of transactions per second. By contrast, for PoW implementations, the transaction throughput is mostly limited by the difficulty of the required proof of work in the protocol. Last, PoA is not susceptible to Sybil attacks since the identities of the nodes are known.

[0049] In some embodiments, the PoA consensus algorithm uses the following formula to decide which nodes are allowed to seal the next block:

$$(\text{Number of consecutive blocks mined per sealer}) = (\text{floor}(\text{Number of Sealers}/2) + 1)$$

[0050] In other embodiments, the transaction will be validated and sealed by the first two available sealers in order to ensure prompt availability of content. In some embodiments, block proposals will not be used. In such embodiments, once a song or media file is requested by a client device, a short buffer of the song or media file is instantly transmitted or streamed. However, the full song or media file will not be piped to the client device until the transaction is finalized on the blockchain, usually within less than a second later. In some embodiments, the playback of a media file is not accounted for until a predetermined amount of the content has been played by the end user. Thus, in cases where an end-user does not consume the content for the predetermined length of time, the initialization of the play may have occurred and some content may be streamed to the end-user, however since they did not consume the content for long enough, the play event will not be sent to the blockchain.

[0051] According to various embodiments, billions of transactions can be made to the blockchain every hour. Consequently, a large amount of data may be stored on the blockchain. Thus, it may become impractical to control storage costs if every node needed to have a full history of the blockchain. Thus, in some embodiments, various different compatible versions of the blockchain client are created. In such embodiments, “lighter” clients will have the same functionality as the regular blockchain client. However, these clients do not need to contain all of the past blockchain history in order to do certain tasks such as validating transactions. Thus, in some embodiments, light clients do not need to download a portion of the blockchain history. In some embodiments, sealers only need to download the latest block's header that was sealed in order to start validating transactions. In some embodiments, sealers are only tasked with validating cryptographic signatures and do not need any history of the blockchain's events. For “heavier” clients, data on the blockchain are pruned by using block headers as references. In some embodiments, lighter clients can use a Merkle Tree to hash data and store the root to reduce the required storage space. However, in some embodiments, parties have the option of running full nodes to have copies of the entire blockchain. In some embodiments, parties can choose whether to keep blockchain data directly on the node or to offload the data into a separate storage solution.

[0052] According to various embodiments, different parties involved with the platform will need different digital infrastructures. For example, DSPs and Music Labels may want to participate in

securing the network using the platform's consensus protocol. However, different parties are free to make their own blockchain clients as long as they follow the platform's consensus protocol specifications.

[0053] In some embodiments, a node can be run by a single computer. In some embodiments, a node can be run across an array of computers. Some architecture choices for running a node can include using load balancers or serverless technologies. The main requirement for each node is that it needs to represent one identity, perhaps by being represented by one or many public keys registered against the participant's identity.

[0054] In some embodiments, servers of the platform provide more services than just being a participant in the network. In some embodiments, the servers are responsible for serving digital media content directly to listeners via a content delivery network (CDN). In such embodiments, the platform will use the anycast methodology with nginx for a distributed set of content servers. In such embodiments, each time a file is served, it is also fingerprinted with the transaction hash or other identifying information. In other embodiments, servers of the platform provide the protocol and tracking services, but do not deliver the content directly.

[0055] In some embodiments, DSPs will be required to change their client side applications. For example, DSPs likely have a separate endpoint for each song on their platform. Thus, in some embodiments, DSPs will need to change the endpoints currently serving the file to one that will make a request to the blockchain network. Once the request is validated, the full song file will be piped to the user by the CDN.

[0056] In some embodiments, in order to cryptographically verify that a user has made a certain action, the blockchain protocol leverages asymmetric cryptography, such as elliptical curve digital signature algorithms (ECDSA). In such embodiments, end-users need to create a signature every time a request is sent. Thus, DSPs must supply their listeners (either on the client's device or using the DSP's own server) with a cryptographic keypair.

[0057] According to various embodiments, due to the large amount of transactions per second handled by DSPs, several methods for scaling are provided. In some embodiments, a consortium blockchain is implemented instead of a public blockchain. When building the consortium blockchain, PoA is used. By staking their identity/reputation, transaction validators are able to quickly approve requests and are only constrained by their own hardware and internet infrastructure.

[0058] In some embodiments, it is not feasible to use a single blockchain to record transactions occurring across the network. In some embodiments, the blockchain can be partitioned into geographic shards. For example, one shard could be responsible for North America while another is responsible for London. In some embodiments, including other embodiments noted in the present disclosure, the blockchain can be partitioned by Label and DSP relationship. For example, a shard can be between StreamCo and Label A while StreamCo has another shard with Label B. In such embodiments of relationship sharding, the participants cannot access raw data from a relationship that they do not belong to.

[0059] In some embodiments, each participant in the blockchain network is required to run a minimum number of nodes spread across the globe. Thus, the platform's bootnodes will geographically group sealers using a routing algorithm. In some embodiments, once a shard is spawned from the main chain, it is periodically merklized and the root hash is taken and stored on the main chain. Periodically, or once the shard is closed, the history of it is added to the main chain. In some embodiments, in the unlikely case of a dispute in a shard, the most recent cryptographically provable data will be used as the truth. In some embodiments, end users are assigned and unassigned from geographic shards by requests made on the main or DSP-Label relationship shards. In some embodiments, once a user is assigned to a certain shard, they cannot make requests in another shard within the same DSP-Label relationship shard.

[0060] In some embodiments, off-chain scaling can be incorporated into the protocol. For example,

state channels, child chains, or sharding chains can all be used in the consensus protocol. The following example illustrates one method for incorporating off-chain scaling. In the example, the platform's sharding implementation uses a main chain transaction to open a relationship. As a note, every media play is counted instead of just the final outcome of the state channel.

[0061] CDN Example: Alice opens up her music application to play music from a DSP. She is assigned to the appropriate geographic shards for her DSP (either existing or newly created if resources are available). Alice can now stream any amount of media content without stressing the main chain. Each stream request is signed by Alice using her private key and can be verified by any sealing node by using her public key.

[0062] Looking at the following use case of her music application one of ordinary skill in the art can see how sharding improves performance: Alice requests to play a song by cryptographically signing a request. The request is verified by at least two sealers before the platform CDN streams music to Alice. The call to the song's address will be recorded to the shard's blockchain. The difference between this method and a conventional call to the blockchain is that the requests won't be committed to the main chain right away. At arbitrary intervals in the song's runtime, a request is automatically made from the client within the shard to get the next part of the song the user is listening to. Data will be piped in chunks to keep track of playback data. If at any point there's a transaction that wasn't signed by Alice or there's a dispute on the number of songs that she played, the most recent cryptographically provable data is committed to the main chain. Then the shard continues or closes and the main chain reassigns all the active users in the shard. If Alice hasn't listened to any music for an arbitrarily set amount of time (as defined when she joined the shard), then she is unassigned from all shards.

[0063] In some embodiments, every song streamed from one of the platform's CDNs will be stamped allowing the original streamer of the content to be identified. In some embodiments, the entire fingerprinting process is broken up into two main components: stamping (applying the fingerprint to the track) and scraping (finding tracks that were originally played from the platform's CDNs and identifying how the content leaked). In some embodiments, the fingerprinting method must remain secret in order to ensure bad actors don't try to remove or distort their identifying fingerprint.

[0064] Although the example above with regard to Alice involves CDN architecture, the protocol with blockchain recordation of play requests and counts pertaining to the same also apply to non-CDN architecture. Transaction requests can be validated and recorded in an analogous manner to the above example except that the delivery of content is provided by DSPs and not directly through the platform (third party play count record keeper and auditor) implementing the protocol. In a non-CDN system, once a transaction is seen by a validator (or sealer), the data within the transaction can be checked for cryptographic validity by verifying the transaction data against the end user's public key. Once this validation step passes, the transaction data is signed by the validator and the resulting signature is written back to the platform along with the associated transaction ID. Signatures are aggregated by transaction and stored in a long term storage solution. All validators are able to read directly from the stream. As a result, validators within the same network cluster will have the ability to capture the same set of signatures that are associated with each specific transaction by associating the data with the transaction through its transaction ID.

[0065] In some embodiments, every node/validator has an API endpoint (e.g., a URL) for the stream. In some embodiments, a stream is similar to a server, but with the difference that a transaction can sit in the stream for an arbitrary amount of time.

[0066] In some embodiments, a playback request is initially formed by an end user. The request format must follow the rules set out by the protocol. The request is then signed by the end user who created the request so the origin of the request can be verified in the future. The signed request and the accompanying data are sent to the platform's stream, again, in a format that is set by the protocol. For example, if the first receiving node of the play request was the DSP, then the DSP can

subsequently send the verified and signed request to the platform and/or label. After the platform and/or label verifies and signs the request, the verified and signed request is submitted back to the DSP. Once two validators sign the request, the transaction is added to the blockchain. In some embodiments, a single transaction can constitute a block in the blockchain. In other embodiments, a block can constitute many transactions.

[0067] In some embodiments, each validator has the option to store a transaction on their own copy of the blockchain at any time. In other embodiments, the validators synchronize their blockchain with other validators.

[0068] In some embodiments, child chains are spawned when a new DSP/label relationship is formed. In some embodiments, each DSP has their own branch child chain from the main chain. However, maintaining raw data on parent chains and child chains can become computationally heavy. Thus, in some embodiments, raw data stays on the child chains while the merklized root nodes of the raw data are propagated up the parent chains.

[0069] In some embodiments, when the data exists in the stream, it can be cryptographically verified by validators on the platform. Each time a request is validated, the transaction data is then signed by the validator. The signature is written back to the stream with a reference to the ID of the request data.

[0070] Each validator can capture the signature of the other validators and keep them in their own ledger. This is done in order to demonstrate that the original transaction was validated by specific validators in the future.

[0071] Depending on the implementation of the data flow, the end user can pass the request directly to a validator instead of writing the request directly to the stream. If valid, the request is then signed by the validator. At this point, the transaction has the original request data and two signatures, one from the end user and one from the initial validator. It is then written to the stream. Now that the data exists in the stream, it can be cryptographically verified by validators on the platform. The original validator who wrote the request to the stream is able to capture new validations coming through the stream. Each time a request is validated, the transaction data is then signed by the validator. The signature is written back to the stream with a reference to the ID of the request data.

[0072] In some embodiments, an initial request is considered valid if: 1) the format of the data follows the protocol established by the platform; and 2) the data is accompanied with a cryptographic signature and the end-user public key. In some embodiments, the end user can use a different format for the request than the protocol. In such embodiments, the DSP or first receiving node can reconfigure the format to align with the platform protocol before it is submitted to the stream.

[0073] In some embodiments, on the output of the stream, the validators listen for unsigned requests and requests that may have been signed by other validators but not themselves. In order to validate the requests that are written to the stream, the validator looks for 3 main pieces of data: the actual request data, the end user's public key that created the request, and a cryptographic signature.

[0074] The request data is verified to have originated from the end user if the public key attached can be used to verify the signature against the request data. Once verified by the validator, the validator signs the request data and writes back to the stream. The data that is written back to the stream consists of a request identifier and the validator's signature, signaling that the transaction is valid.

[0075] As mentioned above, in some embodiments, instead of being written directly to the stream, the request can be passed through to a validator who will then send it either directly to another validator or to the stream itself for distribution. Now when the request enters the stream, it can have 2 or more signatures. One belonging to the end user who originally made the request and one or more belonging to the validators who have validated and signed the request. The validators who signed the request before it made its way to the stream are able to watch the stream to catch signatures on the request that were created by other validators.

[0076] In some embodiments, by stamping files using digital steganography, minimal user data can be hidden within the data that is being sent to the user. Each time a song is streamed or downloaded, a hidden “watermark” can be written to the file. Using digital signal processing, an algorithm can write or identify parts of digital content that are common to every file of that format. For example, every mp3 file will have a point in the song that is a lower pitch than all the other portions of the song. At the common point of the song, an encrypted digital fingerprint of the user can be added.

[0077] In some embodiments, stamp recognition software will extract digital fingerprints by using digital signal processing to deconvolute the original file uploaded from the file that was scraped from the web. If a digital fingerprint is found, it can be decrypted and traced back to a user and time of play. In some embodiments, scraping for fingerprinted content can be broken down into 3 major steps: 1) Build a web crawler for common pirating/streaming sites. 2) Extract and cache audio files. This step requires a custom solution for scraping for each streaming/download site. 3) Run cached audio against the stamp recognition software.

[0078] According to various embodiments, the systems and methods disclosed allow for verification of catalog, ownership, and payment details for media files. In some embodiments, the systems also allow for accruing and accounting of payments. In some embodiments, the systems also allow for depositing the correct amounts in the correct accounts. In some embodiments, the systems provide for equitable, transparent, auditable, and near-real-time payment for creators of musical works and sound recordings (artists), labels, and distribution platforms. In addition, in some embodiments, the systems provide real-time compliance and auditing mechanisms.

[0079] FIG. 1 shows an example diagram of some available user types' relationships, data, and functions in disclosed systems, in accordance with embodiments of the present disclosure. In some embodiments, based on a user's role, they will have different responsibilities on the disclosed platform. In FIG. 1, every user **106** is able to make their own cryptographic key pair for signing data and verifying signatures against given data and public keys.

[0080] In some embodiments, labels **102** are in charge of uploading songs and album metadata to the platform. In some embodiments, labels can additionally approve songs in their catalog to be distributed by DSPs. In some embodiments, label administrators prepare a song or album for the platform by filling in the required metadata, uploading the song file, and assigning stakeholder shares. Upon receiving a valid upload request from a label account, an entry is broadcasted to the blockchain. In response to an artist being credited to a song, the artist now has the option to approve the song, which will make it available to the DSP on the song's release date.

[0081] In some embodiments, the uploaded song is held on a media server and can only be accessed by users who have made a cryptographically valid request to the blockchain to access the content. In some embodiments, the administrator can use the platform web product to view analytical data about song playbacks. In some embodiments, the administrator can break down data by each request, allowing them to track how users are engaging with their content.

[0082] In some embodiments, once an artist **104** gets a contract offer with a label **102**, artist **104** can accept being added to the label **102**. In some embodiments, artist **104** may not be assigned to a label. In such embodiments, artist **104** is able to act as their own label on the platform. When a label **102** uploads a song, it is unconfirmed until the credited artist **104** approves the upload. Artists **104** are allowed to view their own song playback data via the platform but are restricted from viewing any of their colleagues' data.

[0083] In some embodiments, stakeholders **108** for each song are determined before getting finalized on the blockchain. In some embodiments, a percentage of revenue of each song is assigned to stakeholders **108**. In some embodiments, stakeholders **108** are able to view data on songs which they own part of.

[0084] In some embodiments, thanks to DSPs **110**, end-users **106** are given access to songs. Since most DSPs **110** use a subscription model there's functionality to allow a user **106** to listen to any

song on the DSP's platform for a set time period. In some embodiments, the suggested changes that existing DSPs will need to make to their systems in order to leverage the platform's technology are minor.

[0085] In some embodiments, users **112** of a DSP's application should not perceive any difference when streaming songs. Instead, the DSP **110** that created the end user is in charge of managing the user's key. Whether the DSP wants to sign requests using their own servers or their client applications is up to the DSP.

[0086] In some embodiments, latency issues arise when nodes are not sufficiently close, meaning the latency between two nodes goes up as the distance between them increases. By leveraging geographic sharding of the child chains, signal travel time will be reduced. FIG. 2 will help explore latency on the blockchain network. FIG. 2 illustrates an example latency analysis from a song play request to serving fingerprinted content, in accordance with embodiments of the present disclosure. The latency **200** can be broken down into steps, some of which run in parallel.

[0087] At **202**, a song is requested for play. In some embodiments, when an end user plays a song, the request is transmitted to 2 main places: the blockchain shard assigned to the user and the CDN. At **204**, the fingerprinted file is prepared. Steps **206** and **208** deal with sealing the request. In some embodiments, the first 2 out of 3 distinct parties to validate the request will signal that the transaction is sealed. At **206**, the request is validated by the first node. At **208**, the request is validated by the second node. Not visualised here is a third node that is slower than the other two nodes, **206** and **208**, at validating the incoming play request. This slower node can still have its cryptographic signature attached to the play event eventually, however it is not required to seal this play request. At **210**, content is served to the user. In some embodiments, the server pipes the digital media file to the user. As a result of the path outlined in FIG. 2, the following equation for a successful file response can be written:

$$t.sub.latency = \max(t.sub.client \rightarrow CDN, t.sub.client \rightarrow Node\ 1, t.sub.client \rightarrow Node\ 2) + \max(t.sub.prep\ file + t.sub.Node\ 1 \rightarrow CDN, t.sub.Node\ 2 \rightarrow CDN) + t.sub.serve\ file$$

[0088] In some embodiments, privacy of data is important. According to various embodiments, there are two ways for DSPs and labels to ensure that their competitors are not able to view their analytical data: 1) multi-layered sharding and 2) zero knowledge proofs.

[0089] FIG. 3 illustrates an example diagram of multi-layered sharding, in accordance with embodiments of the present disclosure. FIG. 3 illustrates main chain **302**, relationship shards **304** and **308**, as well as geographic shards **306** and **310**. In some embodiments, each Label-DSP relationship is grouped into its own shard **304** or **308**.

[0090] In some embodiments, zero knowledge proofs are also used to ensure data privacy. With zero knowledge proofs, network participants can cryptographically validate a transaction without revealing the data in the transaction. In some embodiments, any data written to the chain should use one-way encryption. This keeps any data on the chain as only readable by the parties who have the key to decrypt the data. In some embodiments, the decryption key on each relationship shard will be shared between the DSP and Label.

[0091] In some embodiments, while having the platform own the servers for the CDN requires the least trust between labels and DSPs, there are alternative tracking methods available. For example, the platform can create a software development kit (SDK) or library to run with the DSPs' applications. In such an example, the primary purpose of the SDK is to provide the DSP with the right cryptography algorithms used to sign and verify transactions. In some embodiments, the DSP will be required to provide a report at regular intervals or by certain triggers, with playback event data and their associated signatures. In some embodiments, the platform will arrange an auditing system where the system creates end-user accounts on the streaming platform and ensures that the plays are being counted according to the protocol. In some embodiments, the benefits of using the SDK include the fact that the DSPs will be happy they can hold onto the content. In some

embodiments, the drawbacks of this method include the fact that the DSPs still control the content on their servers and that Labels can only verify figures by doing a covert audit of the DSP. In some embodiments, the platform will have to write an implementation of the plugin for each of the DSPs' interfaces (web, mobile app, desktop app).

[0092] In some embodiments, another alternative tracking method involves encrypted codecs. In such embodiments the platform can create a custom file type to encrypt a file every time it is accessed. In some embodiments, the DSPs can then hold the encrypted files on their own servers. This encryption method could take in a signed request by the user in order to decrypt the file. In some embodiments, the benefits of this method include the fact that offline interactions can be trustlessly taken into account with this method. In some embodiments, another benefit is the possibility of covertly stamping the user's data to the file in the decryption step. In some embodiments, yet another benefit includes the fact that DSPs are happy they can hold onto the content. In some embodiments, the drawbacks of this method include the fact that it requires talent to research and implement a new standard. In some embodiments, another drawback includes the fact that there is still a vector to steal the contents of the file by accessing the device's cache on which the content is being played. In some embodiments, yet another drawback is that fact that DSPs have to implement the codec standards the platform puts forward.

[0093] FIGS. **4-10** are illustrated to provide more context to the music industry and the role that the platform (and the disclosed systems and methods) plays in licensing and streaming music.

[0094] FIG. **4** shows a diagram **400** of one example of how the music industry is connected, in accordance with embodiments of the present disclosure. In FIG. **4**, composers **402** assign musical works copyright to publisher **404**. Next publisher **404** grants a performance license to performance rights organizations (PROs) **428**. PROs **428** then issues blanket licenses to Radio/TV **424**, Stadiums **422**, and DSPs **418**. Going back to publisher **404**, publisher **404** also gives reproduction licenses to sheet music printers **406**, subpublishing licenses to foreign publishers **408**, sync licenses to movie studios **410**, as well as mechanical licenses to record companies **412**. In some embodiments, the mechanical licenses go through the Harry Fox Agency **416**. Record companies **412** are also assigned sound recording copyrights from performers **420** and then subsequently provide sound exchange **414** with DSPs **418**.

[0095] FIG. **5** shows a diagram **500** of one example of the role of a basic administrator, in accordance with embodiments of the present disclosure. First, administrative dashboard **512** sends a song creation request **502** to server **504**. Next, server **504** creates a contract **506** on blockchain **508**. Last, administrative dashboard **512** analyzes song usage **510** at the blockchain.

[0096] FIG. **6** shows a diagram **600** of one example of a basic data flow of an end-user, in accordance with embodiments of the present disclosure. Diagram **600** begins with client application **612** sending a play song request **602** to platform CDN server **614**, blockchain **608**, and DSP authentication server **616**. Next, DSP authentication server **616** communicates **604** to the blockchain network **608** that the user is indeed allowed to access the specified resource. Next, blockchain **608** broadcasts that the play is ready to be initialized by the CDN **614** to the client application **612**. Last, platform CDN server **614** serves the content to the user **610**.

[0097] FIG. **7** shows a diagram **700** of one example of a media file playback tracking network, in accordance with embodiments of the present disclosure. DSP **702** transmits a content request **704** to blockchain network **706**. Blockchain network **706** then verifies the request **716**. After the request is verified and considered sealed, CDN **720** serves content **718** to the DSP **702** or to the DSP's end-user who made the original playback request. Transactions that are validated are archived **708** for later inspection or audit into a data storage solution **710**. Periodically or upon certain events, a segment of the blockchain data is distilled down to its merkle root hash **714** and that root hash is written to a third party blockchain network **712**. The mechanism for writing to an external blockchain is further explored in FIG. **13**.

[0098] FIG. **8** shows a system diagram of an example of a system **800** for exchanging information

within a network environment, in accordance with some implementations. System **800** includes a variety of different hardware and/or software components which are in communication with each other. In the non-limiting example of FIG. **8**, system **800** includes at least one system server **804**, at least one client device **808**. In some embodiments, system **800** includes at least one digital service provider (DSP) **880**. One example of a DSP is Spotify or iTunes. In some embodiments, system **800** also includes a music label **882**, and/or at least one artist **886**. In some embodiments, music label **882** represents and uploads music from artists **886**, but sometimes, artists **886** do not have a record label and thus upload music directly to system server **804**. Often times, client device **808** streams songs from DSP **880**. System **800** allows for system server **804** to help keep track of what songs are being streamed at client **808** via DSP **880**.

[0099] System server **804** may communicate with other components of system **800**. This communication may be facilitated through a combination of networks and interfaces. System server **804** may handle and process data requests and data transfers from the client device **808** (for direct content delivery models) and DSP **880**. Likewise, system server **804** may return a response to client device **808** after a data request has been processed. For example, System server **804** may retrieve data from one or more databases, such as music label **882** or artist **886**. It may combine some or all of the data from different databases, and send the processed data to one or more client devices or DSPs.

[0100] A client device **808** and DSP **880** may be computing devices capable of communicating via one or more data networks with a server. Examples of client device **808** and DSP **880** include a desktop computer or portable electronic device such as a smartphone, a tablet, a laptop, a wearable device, an optical head-mounted display (OHMD) device, a smart watch, a separate server, etc. Client device **808** and DSP **880** include at least one browser in which applications may be deployed.

[0101] Music label **882** can be a database implemented in a relational or non-relational database management system. In some embodiments, this database can include the contents of one or more client-related databases within a network environment.

[0102] Artist **886** can be an individual artist not associated with a record/music label, or an artist that is associated, but has special needs not fulfilled by the music label. In some embodiments, communication with artists **886** can include edits, modifications, and/or tracked changes information related to songs, albums, or any other streaming media associated with the artist.

[0103] FIG. **9** shows a flowchart of a method **900** for tracking media file playback using CDN architecture, in accordance with embodiments of the present disclosure. Method **900** begins with receiving (**902**) a request to upload a media file and metadata associated with the media file. Next, the media file and metadata associated with the media file are uploaded (**904**) via a blockchain protocol. At **906**, a request to play the media file from a client device or a digital service provider (DSP) platform is received. At **908**, the request to play the media file is validated via the blockchain protocol. At **910**, upon validating the request to play the media file, the media file is transmitted or streamed for playback at the client device or DSP platform. However, a small portion of the beginning of the media file could be streamed to the end user before the request has been validated in the interest of keeping latency low. Last, the number of times the media file is played up to a predetermined length is tracked (**912**) via the blockchain protocol.

[0104] In some embodiments, the blockchain protocol utilizes a proof of authority algorithm. In some embodiments, validating the request includes cryptographically validating the request by one or more authorized sealers (validation nodes) from a plurality of sealers. In some embodiments, the request to play the media file includes a request to the blockchain to access the content at a specified time. In some embodiments, requests to the blockchain can be finalized as soon as they hit the blockchain thereby allowing appending to the blockchain in any order. In some embodiments, a short buffer of the media file is instantly streamed after receiving the request to play the media file, but the rest of the media file is only streamed after the request to play has been

validated. In some embodiments, different versions of the blockchain client can be used for different clients assuming they follow the platform protocol rules.

[0105] FIG. **10** shows a flowchart of a method **1000** for tracking media file playback using non-CDN architecture, in accordance with embodiments of the present disclosure. Method **1000** begins with receiving (**1002**) transaction data from a platform stream. In some embodiments, a stream is a sequence of data elements made available over time. The stream serves as an ingestion point for transaction requests and then outputs the transaction requests to the relevant parties. In some embodiments, the relevant streams can be any number of registered or trusted validation nodes, e.g., DSPs, the platform, labels, or distributors. In some embodiments, validation nodes have to go through a registration process beforehand in order to become trusted. In some embodiments, an end user device creates and signs the transaction data and then transmits the data to the platform stream. The transaction data corresponds to a request to play a media file from an end user. In some embodiments, the transaction data includes a cryptographic signature corresponding to the end user. This is to verify that the initial request came from the end user.

[0106] Next, the transaction data is verified (**1004**). In some embodiments, verifying the transaction data includes verifying: 1) whether the format of the transaction data follows a protocol established by the platform, and 2) whether the transaction data includes a cryptographic signature of the end user that can be verified with a public key corresponding to the end user. At **1006**, the verified transaction data using a cryptographic signature is signed. In some embodiments, the signor is the receiving validation node. In other words, once the transaction request is verified, the receiving validation node adds its own cryptographic signature.

[0107] At **1008**, it is determined whether the transaction data corresponds to a valid blockchain transaction. In some embodiments, a blockchain transaction is considered valid if it has been signed by at least two validation nodes/sealers. If the transaction data corresponds to a valid blockchain transaction, then the valid blockchain transaction is recorded to a blockchain. In some embodiments, the signature just added by the receiving validation node in step **1006** may be the second signature needed to deem the transaction as valid. In some embodiments, the blockchain is the platform's blockchain or a copy of the platform's blockchain. In some embodiments, the blockchain utilizes a proof of work algorithm.

[0108] Last, the transaction data and the cryptographic signature are transmitted (**1010**) to one or more validation nodes. In some embodiments, the transaction data and cryptographic signature are resubmitted to the stream. If the receiving node is the platform, then the other validation nodes may be DSPs, labels, or distributors. If the receiving node is a DSP, then the other validation nodes may be the platform, labels, or distributors. In some embodiments, transmitting the transaction data and the new signature to other nodes propagates the verification protocol, especially if the transaction data received in step **1002** included less than two validation node signatures.

[0109] FIG. **11** illustrates an example non-CDN platform architecture for tracking media file playback, in accordance with embodiments of the present disclosure. End-user **1102** creates and sends a play request to DSP **1104**. After verifying and validating the request, DSP **1104** then streams the content to end-user **1102**. Counts of the play requests of songs are sent by DSP **1104** to the stream, which then sends the requests to a network of validation nodes **1106**. The network of validation nodes **1106** includes platform **1110**, which receives the request counts and in turn propagates the request counts to label **1112** and back to DSP **1104**. In some embodiments, once a transaction request has been validated, it is stored separately in a database as raw captured data **1114**. In some embodiments, the raw captured data is written to storage, e.g., a Structured Query Language (SQL) database, by the platform's archiver. In some embodiments, this raw captured data is then made available to authorized and relevant parties via querying a separate API **1116**. In some embodiments, data on the blockchain stored at platform **1110** is merklized into seemingly random strings of characters called merkle roots. These merkle roots are then committed to a third-party/public external blockchain **1118**, such as an external proof-of-work blockchain. This provides

an additional point of security because the increase in hashing power of an external third-party blockchain increases the cost associated with tampering with the integrity of the blockchain.

[0110] FIG. **12** illustrates an example of a transaction anatomy, in accordance with embodiments of the present disclosure. Transaction **1202** is a data structure that includes request data **1206**, end user signature **1208**, and possibly one or more validation node signatures **1210**, **1212**, or **1214**. In some embodiments, request data **1206** includes request ID **1204**, which is a unique identifier for the transaction itself. For example, as sample request ID **1204** can be generated by concatenating the end user public key with the date and time of the transaction. In some embodiments, request data **1206** includes information regarding the request to play the song. For example, request data **1206** may contain information regarding the title of the song, the time the song was played, who requested the song, and/or the duration of the play of the song. In some embodiments, end user signature **1208** comprises a public-private key pair that uniquely identifies the end user. In some embodiments, the public and private key in the pair share a mathematical relationship. In some embodiments, the signatures for the validation nodes are also the same. Although FIG. **12** illustrates validation nodes as “Party A,” Party B”, and “Party Z,” the parties can refer to any of the following: DSPs, platforms, labels, and distributors.

[0111] FIG. **13** illustrates an example of data flow **1300** of a transaction, in accordance with embodiments of the present disclosure. Data flow **1300** begins with transaction data being created (**1302**) by an end user. The data is then signed (**1304**) by the end user. In some embodiments, the signature is a cryptographic signature that uniquely identifies the end user. Next, the data and the user signature are sent to the stream **1306**. In some embodiments, the stream can be one single stream with access points from end users and validation nodes. In some embodiments, the stream is specific to each validation node. In some embodiments, the stream can be a hierarchy of different streams from different nodes. After the data and user signature hit the stream **1306**, the data and user signature are sent to multiple validation nodes (Part A . . . Party Z). The nodes then verify (**1310**, **1308**) the data and the end user signature. After verifying, the nodes then sign (**1314**, **1316**) the data themselves. Next, the data and the nodes' signatures are then sent to the stream (**1318**, **1320**). In some embodiments, the nodes can optionally collect (**1322**, **1324**) the other signatures attached to the transaction by the other validators.

[0112] FIG. **14** illustrates another example of data flow **1400** of a transaction, in accordance with embodiments of the present disclosure. Data flow **1400** begins with transaction data being created (**1402**) by an end user. The data is then signed (**1406**) by the end user. In some embodiments, the signature is a cryptographic signature that uniquely identifies the end user. Next, the data and the user signature are sent (**1408**) to a specific validation node, Party A. In some embodiments, Party A is a DSP. Party A then reads and verifies the data and end user signature (**1410**). After verifying the data and the end user signature, Party A then signs (**1412**) the data with its own cryptographic signature. Next, the data, the end user signature, and Party A's signature are sent to the stream (**1414**). In some embodiments, the stream can be one single stream with access points from end users and validation nodes. In some embodiments, the stream is specific to each validation node. In some embodiments, the stream can be a hierarchy of different streams from different nodes. After the data, user signature, and Party A's signature hit the stream **1414**, the data, user signature, and Party A's signature are sent to multiple validation nodes (Part B . . . Party Z). The nodes then verify (**1416**, **1418**) the data and the end user signature. After verifying, the nodes then sign (**1420**, **1422**) the data themselves. Next, the data and the nodes' signatures are then sent to the stream (**1424**, **1426**). In some embodiments, the nodes can optionally collect (**1428**, **1430**) the other signatures attached to the transaction by the other validators.

[0113] FIG. **15** illustrates a flow diagram of an example non-CDN platform architecture **1500**, in accordance with embodiments of the present disclosure. An incoming request **1501** is created by an end user, signed by an end user, and then sent to stream **1502**. In some embodiments, request **1501** is a request to play a song. Stream **1502** then passes the request to validator A (**1504**), validator B

(1506), and validator C (1508) to be validated according to the platform's protocol. Once each validator verifies and signs the request, it is then resent to the stream. In some embodiments, once two different validators have signed the request it is considered valid and then the request is sent to the platform archiver 1510 to be stored in a separate database.

[0114] FIG. 16 illustrates another flow diagram of an example non-CDN platform architecture 1600, in accordance with embodiments of the present disclosure. An incoming request 1601 is created by an end user, signed by an end user, and then sent to validator A (1604). In some embodiments, request 1601 is a request to play a song. After validator A verifies the request and signs the request, it then sends the request to stream 1602. Stream 1602 then passes the request to validator B (1606) and validator C (1608) to be validated according to the platform's protocol. Once each validator verifies and signs the request, it is then resent to the stream. In some embodiments, once two different validators have signed the request it is considered valid and then the request is sent to the platform archiver 1610 to be stored in a separate database.

[0115] Various computing devices can implement the methods described. For instance, a mobile device, computer system, etc. can be used for accessing aspects of the network environment by either the client, music label, or DSP. With reference to FIG. 17, shown is a particular example of a computer system that can be used to implement particular examples of the present disclosure. For instance, the computer system 1700 can be used to provide generate artificially rendered images according to various embodiments described above. In addition, the computer system 1700 shown can represent a computing system on a mobile device. According to particular example embodiments, a system 1700 suitable for implementing particular embodiments of the present disclosure includes a processor 1701, a memory 1703, an interface 1711, and a bus 1715 (e.g., a PCI bus). The interface 1711 may include separate input interface 1713 and output interface 1717, or may be a unified interface supporting both operations. When acting under the control of appropriate software or firmware, the processor 1701 is responsible for such tasks such as optimization. Various specially configured devices can also be used in place of a processor 1701 or in addition to processor 1701. The complete implementation can also be done in custom hardware. The interface 1711 is typically configured to send and receive data packets or data segments over a network. Particular examples of interfaces the device supports include Ethernet interfaces, frame relay interfaces, cable interfaces, DSL interfaces, token ring interfaces, and the like.

[0116] In addition, various very high-speed interfaces may be provided such as fast Ethernet interfaces, Gigabit Ethernet interfaces, ATM interfaces, HSSI interfaces, POS interfaces, FDDI interfaces and the like. Generally, these interfaces may include ports appropriate for communication with the appropriate media. In some cases, they may also include an independent processor and, in some instances, volatile RAM. The independent processors may control such communications intensive tasks as packet switching, media control and management.

[0117] According to particular example embodiments, the system 1700 uses memory 1703 to store data and program instructions and maintain a local side cache. The program instructions may control the operation of an operating system and/or one or more applications, for example. The memory or memories may also be configured to store received metadata and batch requested metadata.

[0118] Because such information and program instructions may be employed to implement the systems/methods described herein, the present disclosure relates to tangible, machine readable media that include program instructions, state information, etc. for performing various operations described herein. Examples of machine-readable media include hard disks, floppy disks, magnetic tape, optical media such as CD-ROM disks and DVDs; magneto-optical media such as optical disks, and hardware devices that are specially configured to store and perform program instructions, such as read-only memory devices (ROM) and programmable read-only memory devices (PROMs). Examples of program instructions include both machine code, such as produced by a compiler, and files containing higher level code that may be executed by the computer using

an interpreter.

[0119] FIG. **18** shows a linked list data structure **1800** represented with three data segments **1802**, **1804**, and **1806**. Conventional blockchains are structured similar to linked lists such that each segment of data refers to a previously created segment of data. For example, data segment **1806** contains a set of arbitrary data. In order for data segment **1806** to become a part of the entire data set, it contains a reference to the data segment that preceded it. In this case data segment **1804** comes before data segment **1806**, therefore data segment **1806** contains a reference to data segment **1804**. In some embodiments, the first entry into a linked list, FIG. **18** the data segment **1802**, does not have a reference to any other data segments.

[0120] FIG. **19** shows a conceptual example of a blockchain **1900**. Each block (**1902**, **1904**, and **1906**) contains a set of data within it. This data can be arbitrarily set, however, it is traditionally a list of transactions. In a similar spirit to the linked list in FIG. **18**, each block has a reference to the block that preceded it. One distinguishing trait of blockchains as a data structure is that they contain cryptographic proofs that the data that they hold in each block has not been tampered with.

[0121] To ensure that historic data on the platform's network as a whole has not been tampered with, periodically segments of the data are aggregated and merklized into seemingly random strings of characters called "merkle roots." FIG. **20** illustrates an example of this merklization process **2000**. In FIG. **20**, segments of data **2004** on platform blockchain **2008** are aggregated and merklized into merkle roots **2006**. These merkle roots of the set of data are then written to a third-party blockchain **2002** that could be backed by any consensus algorithm such as Proof of Work or Proof of Stake. This serves as a supplemental point of security or audit point. The motivation behind creating audit points that lie outside of the platform's blockchain **2008** is so any attacks performed on the platform's blockchain would be more expensive to undertake since the third party network **2002** would need to be attacked as well. For example, FIG. **20** shows an example of merklizing of a portion of the platform's data set **2004**. Given a set of data that is claimed to be from a specific segment of the platform's blockchain, the veracity can be concluded by merklizing the set of data and ensuring it produces the same merkle root that exists on the third party blockchain **2002**.

Streaming Fraud Detection

[0122] The techniques disclosed above allow for reliable play count tracking using blockchain. However, blockchain technology can also be used to detect fraudulent behavior. As different types of media, such as movies, music, and games, are becoming increasingly streamed, new methods have been created to track their usage. However, many new tracking methods are prone to usage fraud.

[0123] For example, some platforms pay rights holders based on how often their content is streamed. Consequently, rights holders are incentivized to inflate their play counts on those platforms to maximize their payout. If the rights holders, or an entity acting on behalf of the rights holders, can manage to set up an automated system to bolster their play counts on one platform, e.g., a "streaming farm," they can artificially inflate the play count, thereby fraudulently increasing their payouts.

[0124] One problem is that because of the way a streaming farm is built, it is difficult to tell the difference between a real listener and a bot (for fraud). Some streaming farms can even generate 10,000 different "users" for the purposes of inflating play counts. Managers, agencies, and even artists are thus sometimes incentive to inflate their own play counts in order to receive higher payouts. Generally, if these rights holders actually go through the trouble of setting up fraudulent automated play count inflation systems on one system, chances are they are likely to use the same system on different platforms as well. Since data for how an artist is listened to is usually based on a pattern, pattern recognition can be very important in determining fraudulent behavior. Techniques and mechanisms of the present disclosure provide for an analyzer that uses a combination of blockchain technology and machine learning to identify fraudulent streaming activity across

multiple platforms.

[0125] In many instances, a streaming farm utilizes one bot running on three or four different streaming platforms. For example, if the platform is Spotify, it may be difficult or even impossible to distinguish this bot from a human user. However, if a third party is allowed to observe the streaming patterns of this bot across multiple different platforms, including Spotify, then it may be possible to catch the exact same pattern used by the bot for multiple artists.

[0126] Streaming fraudsters are generally careful not to drive just their own music play counts. This is because it would seem unnatural for an artist to all of a sudden have a spike in their play counts, which would usually flag the account as a possible fraudulent account. Instead, in streaming farms, bots are carefully designed to promote many different artists in order “hide” the sudden increase in play counts for the artist that they are really trying to artificially inflate. For example, if a streaming farm's intended inflated target is Arianna Grande, then in order to artificially drive Arianna Grande's play count by 100,000, for example, a bot would likely need to drive other artist's play count by 900,000 in order to bury the artificial increase for Arianna Grande. Since many different artists are inflated, with Arianna Grande accounting for only 10% of the artificial inflation, it becomes difficult to single out Arianna Grande, or agents of Arianna Grande, as the main culprit.

[0127] This technique of burying the intended target is a strategy for streaming farms to prevent identification of bots using identification keys. If many different “keys,” which theoretically could be linked to many different users, all inflate the play count, then a particular bot could not be identified using a particular identification key. Thus, there is a need for a way to identify bots without using unique keys. Using blockchain technology, the techniques and mechanisms of the present disclosure are able to identify bots by identifying their listening patterns across multiple platforms.

[0128] According to various embodiments, streaming services can send streaming data to a “third party escrow” agent, which allows the analysis of streaming patterns across multiple platforms. This eliminates the concern of streaming platforms needing to send streaming data, which can be potentially sensitive and/or confidential information, to competing streaming platforms in order to compare and find fraudulent streaming patterns. In some embodiments, the third party escrow agent then utilizes zero-knowledge proofs to determine fraudulent activity. Zero-knowledge proofs are when two parties prove they have the same data by giving the same hash, without ever needing to give away the actual data. Thus, in some embodiments, actual raw data is not sent to the third party escrow agent, but instead, only hashes of the raw data need to be sent.

Streaming Fraud Detection Details

[0129] According to various embodiments, streaming data from the streaming platforms is posted to the blockchain. This data in turn is read from the blockchain and then used as input in a machine learning model. By its nature, data that exists on the blockchain in a linear fashion is very similar to a linked list data structure. FIG. 21 illustrates an example of a linked list, according to embodiments of the present disclosure. Linked list **2100** includes a first element **2102**, a second element **2104**, and a third element **2106**. Linked list **2100** can hold anywhere from one to many data elements.

[0130] Similarly, the blocks on the blockchain can hold one to many streaming events. In some embodiments, each event can have, but is not limited to, metadata fields corresponding to different pieces of information. For example, the metadata fields may contain information regarding the streamed content itself, details about the user streaming the content, or details about the specific stream itself, e.g., the time of the stream. FIG. 22 illustrates a representation of an example streaming event in JSON format, according to embodiments of the present disclosure.

[0131] FIG. 23 is a block diagram showing component parts of a fraud detection system, in accordance with embodiments of the present disclosure. System **2300** includes blockchain **2302**. In some embodiments, streaming platforms, e.g., Spotify, send data to system **2300** by appending streaming data, or hashes of streaming data, to blockchain **2302**. Compute module **2304**, which has

connectivity to blockchain **2302**, is configured to read from blockchain **2302** and write blockchain data **2306** to the inputs of machine learning module **2308**, which runs a trained machine learning model.

[0132] In some embodiments, compute module **2304** is a linux server hosted on a cloud. In some embodiments, compute module **2304** runs a programming language, such as C++ or Javascript, to parse through each of the events in blockchain data **2306** and transform the appropriate fields into a format that is expected by machine learning module **2308**. In some embodiments, compute module **2304** then sends the transformed data to machine learning module **2308**.

[0133] In some embodiments, once machine learning module **2308** determines a stream to be fraudulent **2310**, a response is sent back to compute module **2304**. In some embodiments, compute module **2304** then takes each event that was determined to be fraudulent and inserts it into a data store as shown in FIG. **24**. FIG. **24** shows an example table containing suspected fraudulent streams, in accordance with embodiments of the present disclosure. In some embodiments, data store **2400** has a table **2402**, which stores a list of the suspected fraudulent events. In some embodiments, data store **2400** is a SQL instance. In some embodiments, the data in table **2400** is completely separate from data in blockchain **2302**. As shown in FIG. **24**, table **2402** may include fields such as platform **2404**, user ID **2406**, stream ID **2408**, and song ID **2410**.

[0134] Once suspected fraudulent streams are identified and input into table **2402** in data base **2400**, the information must be collected by labels or DSPs. There are multiple ways that a label or DSP can collect the suspected fraudulent plays. In some embodiments, one way is through an API that has a server with connectivity to data store **2400**. In such embodiments, the label or DSP is given access to this API. In such embodiments, the API server is the interface between the label or DSP and data store **2400**. In such embodiments, the label or DSP is able to query the API server for the data that pertains to their service or the tracks they own, respectively. In other embodiments, the label or DSP is given direct access to data store **2400**. In such embodiments, in the case where data store **2400** is a SQL instance, the label or DSP is then given the connection details of the instance plus a SQL user with the ability to read the data.

Machine Learning Overview

[0135] According to various embodiments, some important components in a fraudulent streaming detection system are a layered model **2308** (machine learning module **2308**), a social pool **2312**, and a DSP trend pool **2314**. In some embodiments, social pool **2312** and DSP trend pool **2314** are data sources that are used as inputs to the machine learning algorithms that make up the layered model **2308**.

[0136] According to various embodiments, the layers of layered model **2308** are each assigned a machine learning algorithm. In some embodiments, each layer focuses on a group of playback data points such as: song name, artist name, producing company, artist manage, genre, playback length, and inclusion in playlists. For example, layer A of model **2308** might be specifically used for Time Series Analysis using an ARIMA algorithm to forecast future stream counts for the artist of the playback event. As another example, Layer B might contain a K-Means clustering algorithm that clusters based on user playback length and account age.

[0137] According to various embodiments, each output of each layer, such as the predictions and clusterings, are aggregated and built-upon by the next layer in layered model **2308**. In order words, Layer N-1 feeds into Layer N and so on until all layers have seen the data. In some embodiments, once the playback data has been passed through all layers, the streaming playback data is ranked by the system on how likely it is to have been involved (streamed) from a stream farm.

DSP Trend Pool

[0138] According to various embodiments, DSP Trend Pool **2314** utilizes streaming data from many different DSPs, as opposed to in-house stream fraud detection systems that are limited to the DSPs own data. Since DSPs keep usage data private, no single DSP can develop algorithms that use data from multiple platforms. Because of this, DSP Trend Pool **2314** provides unique

advantages over in-house streaming fraud detection systems by collecting trends from multiple DSPs, which can be collected and subsequently used as input for layered model **2308** to generate predictions that are more accurate than any single DSP can achieve by itself.

[0139] In some embodiments, DSP Trend Pool **2314** plays an important role in the layers focused on stream count outlier detection. FIG. **25** is a diagram of components that go into a DSP Trend Pool, in accordance with embodiments of the present disclosure. In some embodiments, layered model **2308** can look at, for example, stream counts **2502** for Artist A on DSP A, and compare it to the artist's growth **2504** on DSP B.

[0140] In some embodiments, layered model **2308** will be able to detect if there are discrepancies between the trends across the DSPs. For example, one would expect that if an artist is being stream farmed, the stream farm will not be running on all DSPs. Rather, the stream farm is most likely running on a single, or very few, platforms. According to various embodiments, layered model **2308** coupled with DSP Trend Pool **2314**, can employ a Stream Count Outlier Detection layer, which will flag these discrepancies as high probability for stream fraud.

[0141] In some embodiments, the plays are generated on a DSP that's separate from the system. The system extracts data from the blockchain by using an authorized computing device such as a virtual machine or bare metal server. In some embodiments, the system starts at the genesis block and extracts the data either one block at a time or a batch of blocks at once. In some embodiments, the system extracts data by first verifying the signatures attached to each transaction and each block. Once the system has cryptographically validated the transaction(s), it parses the message to find relevant play data. In some embodiments, the play data can include the track that was played, the time the track was played, the streaming duration, the current position in the stream, user interaction events, and user identifier among other data points. In some embodiments, the system takes the data and populates it into a cache or database that can be used for further querying in the future.

[0142] In some embodiments, the system uses the data about the track, artist, or user associated with the transaction to query an external cache or database. One example of querying is using the newly invoked transaction's user data to do a relational lookup using SQL to find how many times this user has played a song from this artist in a given time frame.

The Social Pool

[0143] According to various embodiments, social pool **2312** is also a source of data curated for machine learning model **2308**, as shown in FIG. **26**. Like DSP Trend Pool **2314**, social pool **2312** is used by layered model **2308** as input to its layers. In some embodiments, social pool **2312** reflects social trends data. In some embodiments, the goal of social pool **2312** is to provide layered model **2308** with a second perspective of song or artist popularity. With this second perspective, layered model **2308** is able to detect discrepancies between the social popularity of a song and the raw stream count number of that song. For example, in FIG. **26**, an outlier is detected in the popularity of Song A **2602** from DSP **2** when compared the popularity of Song A for other DSPs. In some embodiments, social pool **2312** is used as additional input to DSP Trend Pool **2314**, instead of direct input into layer model **2308**. In some embodiments, as shown in FIG. **27**, social pool **2312** pulls in song and artist name chatter from Twitter, Shazam, Google Trends, and other social media to identify trends in songs and artists.

[0144] In some embodiments, the plays of tracks are recorded on the system's blockchain. The system keeps a list of artists that it has received plays for in a separate database that acts as a cache for quick lookups when compared to the querying speed on a blockchain. In some embodiments, when the system sees an artist with no record, it uses the artist's name and some of their tracks' unique ISRCs to create an entry in the cache for the artist. The reason ISRCs of their tracks are required is to handle cases where two different artists have the same name.

[0145] In some embodiments, once that artist is in the database, the system uses their name and their song names to find them on other platforms. Some data extraction techniques from these other

platforms include using APIs that are available to make HTTP requests and receive data formatted in a format like JSON with the artist's data. In such embodiments, this data could include things like their following on the particular platform, the number of likes they get on average when posting on the platform, or other fan engagement information. In some embodiments, the system takes the data from these platforms and stores it into a database for querying by the systems that analyze streaming behavior.

[0146] In some embodiments, once an artist's social data is in the system's database, it may be normalized by performing various processes. For example, the analysis might be interested in the velocity of followers gained on a certain platform so one transformation process could take the raw follower count for the current week and subtract the previous week from it to get the gain in followers. Then, the system takes the difference in the follower count and divides it by the total followers to get the percentage increase. A similar process is conducted on all of the other platforms that had data extracted from them by the system. The percentage gain is compared on all the platforms and a total play count gain from the previous week is predicted by the system.

[0147] According to various embodiments, the system creates a relationship between social media activity and how they relate to stream counts on certain platforms. For example, if an artist has 10,000 followers on Social Co, they could be expected to get 100,000 streams on a new song release they put out. If the system predicts the artist should only get 100,000 streams on the first day but parses a materially larger amount of plays from the blockchain data, like 1,000,000 streams on the first day, the system would use this as one of many signals to determine if the artist's stream count has been fraudulently boosted.

The Layered Model

[0148] According to various embodiments, layered model **2308** is a collection of machine learning algorithms that are specifically chosen and tuned for music streaming data, as shown in FIG. **28**. In some embodiments, each layer in layered model **2308** has two parts: 1) A machine learning algorithm **2802** that is chosen based on the type of playback metadata it handles, and 2) a weight **2804** that is determined by how important this layer is in predicting a stream fraud event.

[0149] In some embodiments, the machine learning algorithms for each layer are chosen during the training of the layered model **2308**. During the training, features of the music streaming metadata, for example, user playback length (how long the user listened to the song), are fed into many different machine learning algorithms, and the algorithms that perform best on correctly predicting streaming fraud for that feature (e.g. playback length), are chosen as the algorithm for that particular layer. It is important to note that, according to various embodiments, since the playback streaming data that is received from different DSPs has many different features (i.e. columns) associated with the data, no single machine learning algorithm will be able to achieve the best results on its own. For example, there are some features in streaming data that are better suited for Supervised Learning/Classification algorithms, and there are some features that are better suited for Unsupervised Learning/Clustering algorithms. In some embodiments, layered model **2308** uses a combination of models to achieve the best performance on music streaming data.

[0150] According to various embodiments, layered model **2308** is built to be flexible. FIG. **29** illustrates a layered model with composable layers, in accordance with embodiments of the present disclosure. As shown in FIG. **29**, layers are composable, meaning that if layered model **2308** discovers that Layer A **2902** performs better when fed data from Layer B **2904**, the model can reorganize itself. In addition, if layered model **2308** discovers that Layer C **2906** is overfitting the data or is leading to worse performance, the model can reweight itself accordingly.

[0151] FIG. **30A**-FIG. **30C** illustrate an example machine learning model architecture, in accordance with embodiments of the present disclosure.

[0152] Although many of the components and processes are described above in the singular for convenience, it will be appreciated by one of skill in the art that multiple components and repeated processes can also be used to practice the techniques of the present disclosure.

[0153] While the present disclosure has been particularly shown and described with reference to specific embodiments thereof, it will be understood by those skilled in the art that changes in the form and details of the disclosed embodiments may be made without departing from the spirit or scope of the disclosure. It is therefore intended that the disclosure be interpreted to include all variations and equivalents that fall within the true spirit and scope of the present disclosure.

Claims

1. A method for detecting fraudulent streaming activity, the method comprising: obtaining trend data from a plurality of streaming platforms, the trend data including access count activity for streaming media; generating a trend pool using the trend data from the plurality of streaming platforms; identifying a first plurality of streams associated with a first platform; analyzing the first plurality of streams using the trend pool, wherein the trend pool allows analysis of streaming patterns across the plurality of streaming platforms to identify discrepancies between trends across platforms to suggest streaming fraud; and Identifying a subset of the first plurality of streams as potentially fraudulent.
2. The method of claim 1, wherein streaming data is extracted from a blockchain to obtain trend data.
3. The method of claim 1, wherein the streaming data is processed using a machine learning model.
4. The method of claim 3, wherein the machine learning model takes into account user engagement data comprising gyroscope orientation, battery percentage, current city, track label, track distributor, and stream duration.
5. The method of claim 3, wherein the streaming data comprises hashes instead of raw data for zero knowledge proofs.
6. The method of claim 3, wherein the machine learning model includes multiple machine learning algorithms for different attributes of streaming media.
7. The method of claim 3, wherein the streaming data corresponds to other information comprising: device battery information, operating system, and user interaction data.
8. The method of claim 3, wherein the machine learning model utilizes a social trend pool or a DSP trend pool as additional input.
9. The method of claim 1, wherein the first streaming platform is notified that the subset of the first plurality of stream is potentially fraudulent.
10. A system for detecting fraudulent streaming activity, the system comprising: an input interface configured to obtaining trend data from a plurality of streaming platforms, the trend data including access count activity for streaming media; a processor configured to generate a trend pool using the trend data from the plurality of streaming platforms, identify a first plurality of streams associated with a first platform, analyze the first plurality of streams using the trend pool, wherein the trend pool allows analysis of streaming patterns across the plurality of streaming platforms to identify discrepancies between trends across platforms to suggest streaming fraud; and an output interface configured to send an indication that a subset of the first plurality of streams are potentially fraudulent.
11. The system of claim 10, wherein streaming data is extracted from a blockchain to obtain trend data.
12. The system of claim 10, wherein the streaming data is processed using a machine learning model.
13. The system of claim 12, wherein the machine learning model takes into account user engagement data comprising gyroscope orientation, battery percentage, current city, track label, track distributor, and stream duration.
14. The system of claim 12, wherein the streaming data comprises hashes instead of raw data for zero knowledge proofs.

- 15.** The system of claim 12, wherein the machine learning model includes multiple machine learning algorithms for different attributes of streaming media.
- 16.** The system of claim 12, wherein the streaming data corresponds to other information comprising: device battery information, operating system, and user interaction data.
- 17.** The system of claim 12, wherein the machine learning model utilizes a social trend pool or a DSP trend pool as additional input.
- 18.** The system of claim 10, wherein the first streaming platform is notified that the subset of the first plurality of stream is potentially fraudulent.
- 19.** A system for detecting fraudulent streaming activity, the system comprising: means for obtaining trend data from a plurality of streaming platforms, the trend data including access count activity for streaming media; means for generating a trend pool using the trend data from the plurality of streaming platforms; means for identifying a first plurality of streams associated with a first platform; means for analyzing the first plurality of streams using the trend pool, wherein the trend pool allows analysis of streaming patterns across the plurality of streaming platforms to identify discrepancies between trends across platforms to suggest streaming fraud; and means for Identifying a subset of the first plurality of streams as potentially fraudulent.
- 20.** The system of claim 19, wherein streaming data is extracted from a blockchain to obtain trend data.
-